Your first analyst task, done in pandas

1. Load and look

Know your data in 60 seconds

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Read the file

→ (8, 13)

```
df = pd.read_csv("sales.csv")
```

2 Rows and columns

→ (8, 13)

```
df.shape
```

3 First 3 rows

→ 3 rows x 13 cols

```
df.head(3)
```

4 Type of one column

→ float64

```
df["sales"].dtype
```

5 Average sale

→ 800

```
df["sales"].mean()
```

1. Load and look

Know your data in 60 seconds

6 How many cities

→ 4

```
df["city"].nunique()
```

7 Orders per region

```
→ North 2, South 2, West 2, ...
```

```
df["region"].value_counts()
```

8 Blanks in the file

→ 3

```
df.isna().sum().sum()
```

9 Two random rows

→ 2 rows x 13 cols

```
df.sample(2, random_state=1)
```

Interview trap: df.shape has no brackets. It is an attribute, not a method.

2. Select rows and columns

Get exactly what you need

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 One column

→ Pune, Pune, Indore, ...

```
df["city"]
```

2 Two columns

→ 8 rows x 2 cols

```
df[["city", "sales"]]
```

3 One cell by label

→ Pune

```
df.loc[0, "city"]
```

4 One cell by position

→ Indore

```
df.iloc[2, 3]
```

5 Rows with sales over 1000

→ 2 rows: Indore

```
df[df["sales"] > 1000]
```

2. Select rows and columns

Get exactly what you need

6 Two conditions

→ 1 row x 13 cols

```
df[(df["region"] == "South") & (df["qty"] > 5)]
```

7 Value in a list

→ 3 rows x 13 cols

```
df[df["city"].isin(["Pune", "Jaipur"])]
```

8 Same filter, readable

→ 2 rows x 13 cols

```
df.query("sales > 1000")
```

9 City of the biggest sale

→ Indore

```
df.loc[df["sales"].idxmax(), "city"]
```

Remember: & and | need brackets around each condition, or pandas throws an error.

3. Clean text

Messy names, fixed

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Remove extra spaces

→ amit

```
df["name"] = df["name"].str.strip()
```

2 Proper case

→ Divya

```
df["name"] = df["name"].str.title()
```

3 Small letters

→ cancelled

```
df["status"].str.lower()
```

4 Contains a word

→ 2

```
df["email"].str.contains("divya")
```

5 Text before the @

→ amit, amit, divya, ...

```
df["email"].str.split("@").str[0]
```

3. Clean text

Messy names, fixed

6 First 3 letters → Pen, Mou, Pen, ...

```
df["product"].str[:3]
```

7 Length of text → 13, 13, 14, ...

```
df["email"].str.len()
```

8 Replace a word → Cancel

```
df["status"].str.replace("Cancelled",
"Cancel")
```

9 Join two columns → Pen - Cancelled

```
df["product"] + " - " + df["status"]
```

Watch out: .str only works on text. Run df.dtypes first.

4. Missing values

Blanks break every average

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Blanks per column

→ region 1, city 1, sales 1

```
df.isna().sum()
```

2 Rows with any blank

→ 2 rows x 13 cols

```
df[df.isna().any(axis=1)]
```

3 Drop rows missing sales

→ 7 rows x 13 cols

```
df.dropna(subset=["sales"])
```

4 Fill blank sales with 0

→ 0

```
df["sales"].fillna(0)
```

5 Fill with the average

→ 800

```
df["sales"].fillna(df["sales"].mean())
```

4. Missing values

Blanks break every average

6 Fill blank city → Unknown

```
df["city"].fillna("Unknown")
```

7 Copy the value above → West

```
df["region"].ffill()
```

8 Count filled cells → 7

```
df["sales"].count()
```

9 Share of blanks → 0.125

```
df["sales"].isna().mean()
```

Interview trap: count() skips blanks, len(df) does not. 7 vs 8 here.

5. Duplicates

Double counting kills reports

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 How many duplicate rows

→ 1

```
df.duplicated().sum()
```

2 Remove them

→ 7

```
df = df.drop_duplicates()
```

3 Duplicates in one column

→ 2

```
df.duplicated(subset=["email"]).sum()
```

4 Keep the latest per email

→ 5 rows x 13 cols

```
df.drop_duplicates(subset=["email"],
keep="last")
```

5. Duplicates

Double counting kills reports

5 See every repeat → 4 rows x 13 cols

```
df[df.duplicated(subset=["email"],
keep=False)]
```

6 Unique cities → 4

```
df["city"].dropna().unique()
```

7 Renumber the rows → 0 to 6

```
df = df.reset_index(drop=True)
```

Remember: drop_duplicates() returns a copy. Save it back with df = ...

6. Dates and types

Turn text into real dates

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Text to date

→ datetime64

```
df["date"] = pd.to_datetime(df["date"])
```

2 Month number

→ 1, 1, 2, ...

```
df["date"].dt.month
```

3 Day name

→ Wednesday, Monday, Saturday...

```
df["date"].dt.day_name()
```

4 Month label

→ Jan 2024, Jan 2024, Feb 202...

```
df["date"].dt.strftime("%b %Y")
```

6. Dates and types

Turn text into real dates

5 **Orders in March** → 4 rows x 13 cols

```
df[df["date"].dt.month == 3]
```

6 **Days since first order** → 0, 12, 24, ...

```
(df["date"] - df["date"].min()).dt.days
```

7 **Number stored as text** → 10, NaN

```
pd.to_numeric(pd.Series(["10", "x"]),
errors="coerce")
```

8 **Whole numbers** → 5, 2, 20, ...

```
df["qty"].astype(int)
```

Interview trap: errors="coerce" turns bad values into NaN instead of crashing.

7. New columns

Revenue, flags, bands

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Revenue

→ 500, 900, 1200, ...

```
df["revenue"] = df["qty"] * df["price"]
```

2 Yes / no flag

→ No, No, Yes, ...

```
df["big"] = np.where(df["revenue"] > 1000,
"Yes", "No")
```

3 Price bands

→ Low, Mid, Low, ...

```
pd.cut(df["price"], bins=[0, 100, 500,
1000], labels=["Low", "Mid", "High"])
```

4 Map codes to names

→ D, D, C, ...

```
df["status"].map({"Delivered": "D",
"Cancelled": "C", "Returned": "R"})
```

7. New columns

Revenue, flags, bands

5 **Round to 1 decimal** → 166.7, 300, 400, ...

```
(df["sales"] / 3).round(1)
```

6 **Cap extreme values** → 5, 2, 20, ...

```
df["qty"].clip(upper=20)
```

7 **Running total** → 500, 1400, 2600, ...

```
df["revenue"].cumsum()
```

8 **Share of total** → 6.2, 11.2, 15, ...

```
(df["revenue"] / df["revenue"].sum() *
100).round(1)
```

Faster: np.where beats apply(lambda) on big files. Same answer.

8. Group and summarise

SUMIF, COUNTIF, AVERAGEIF in one line

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Sales by region

→ East 300, North 1400, South...

```
df.groupby("region")["sales"].sum()
```

2 Average by region

→ East 300, North 700, South ...

```
df.groupby("region")["sales"].mean()
```

3 Orders per city

→ Indore 2, Jaipur 1, Lucknow...

```
df.groupby("city").size()
```

4 Many numbers at once

→ 4 rows x 3 cols

```
df.groupby("region")["sales"].agg(["sum",
"mean", "count"])
```

8. Group and summarise

SUMIF, COUNTIF, AVERAGEIF in one line

5 **Named results**

→ 4 rows x 2 cols

```
df.groupby("region").agg(total=("sales", "sum"),
orders=("order_id", "count"))
```

6 **Biggest region first**

→ South 3000, North 1400, Wes...

```
df.groupby("region")
["sales"].sum().sort_values(ascending=False)
```

7 **Unique customers per region**

→ East 1, North 1, South 1, ...

```
df.groupby("region")["customer_id"].nunique()
```

8 **Each row's share of its region**

→ 0.3571, 0.6429, 0.4, ...

```
df["sales"] / df.groupby("region")
["sales"].transform("sum")
```

Interview trap: groupby drops blank keys. The order with no region vanished.

PANDAS NOTES

9. Pivot and reshape

Excel pivots, in code

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 **Pivot: region x month** → 4 rows x 3 cols

```
df.pivot_table(index="region", columns="month",
values="sales", aggfunc="sum", fill_value=0)
```

2 **With totals** → 5 rows x 1 cols

```
df.pivot_table(index="region", values="sales",
aggfunc="sum", margins=True)
```

3 **Count table** → 4 rows x 3 cols

```
pd.crosstab(df["region"], df["status"])
```

4 **Wide to long** → 16 rows x 3 cols

```
df[["order_id", "sales", "qty"]].melt(id_vars="order_id")
```

5 **Grand total** → 5600

```
df.pivot_table(index="region", values="sales",
aggfunc="sum", margins=True).loc["All", "sales"]
```

Remember: fill_value=0 shows 0 instead of NaN in empty pivot cells.

10. Join tables

VLOOKUP, without the pain

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Bring in the segment

→ 8 rows x 14 cols

```
df.merge(customers, on="customer_id",
how="left")
```

2 Only matching rows

→ 7 rows x 14 cols

```
df.merge(customers, on="customer_id",
how="inner")
```

3 Who has no segment?

→ 1

```
df.merge(customers, on="customer_id",
how="left")["segment"].isna().sum()
```

10. Join tables

VLOOKUP, without the pain

4 See where rows came from

→ both 7, left_only 1, right_...

```
df.merge(customers, on="customer_id",
how="outer", indicator=True)
["_merge"].value_counts()
```

5 Stack two files

→ 16 rows x 13 cols

```
pd.concat([df, df], ignore_index=True)
```

6 Different column names

→ 7 rows x 15 cols

```
df.merge(customers.rename(columns=
{"customer_id": "cid"}),
left_on="customer_id", right_on="cid")
```

Interview trap: a left join can ADD rows if the right table has repeats. Check len() after.

11. Sort and rank

Top sellers, bottom sellers

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Highest sales first

→ 1800

```
df.sort_values("sales", ascending=False)
```

2 Two columns

→ 8 rows x 13 cols

```
df.sort_values(["region", "sales"],
ascending=[True, False])
```

3 Top 3 orders

→ [1800, 1200, 900]

```
df.nlargest(3, "sales")
```

4 Bottom 2

→ [300, 450]

```
df.nsmallest(2, "sales")
```

11. Sort and rank

Top sellers, bottom sellers

5 **Rank, biggest = 1**

→ [7, 5.5, 5.5]

```
df["sales"].rank(ascending=False)
```

6 **Rank with no gaps**

→ [6, 5, 5]

```
df["sales"].rank(method="dense",
ascending=False)
```

7 **Best order per region**

→ 4 rows x 13 cols

```
df.sort_values("sales").groupby("region").ta
il(1)
```

Remember: method="dense" gives 1, 2, 2, 3. The default gives 1, 2.5, 2.5, 4.

12. Time series

Month on month, the analyst way

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 Monthly total

→ Jan 1400, Feb 3000, Mar 1200

```
df.resample("MS", on="date")["sales"].sum()
```

2 Change vs last month

→ Jan NaN, Feb 1600, Mar -1800

```
df.resample("MS", on="date")
["sales"].sum().diff()
```

3 Growth %

→ Jan NaN, Feb 114.3, Mar -60

```
(df.resample("MS", on="date")
["sales"].sum().pct_change() * 100).round(1)
```

12. Time series

Month on month, the analyst way

4 **Previous value** → NaN, 500, 900, ...

```
df["sales"].shift(1)
```

5 **2-order moving average** → NaN, 700, 1050, ...

```
df["sales"].rolling(2).mean()
```

6 **Running total by region** → 500, 1400, 1200, ...

```
df.groupby("region")["sales"].cumsum()
```

Interview trap: resample needs a datetime column. Convert first.

PANDAS NOTES

13. Export your work

Hand it to your manager

Data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 **Save as CSV** → True

```
df.to_csv("out/report.csv", index=False)
```

2 **Save as Excel** → True

```
df.to_excel("out/report.xlsx", sheet_name="Sales",
index=False)
```

3 **Two sheets, one file** → 1 file, sheet All

```
with pd.ExcelWriter("out/two.xlsx") as w: df.to_excel(w,
sheet_name="All")
```

4 **Round before you send** → True

```
df.round(2).to_csv("out/clean.csv", index=False)
```

5 **Check what you saved** → (8, 13)

```
pd.read_csv("out/report.csv").shape
```

Remember: index=False, or your file gets an extra unnamed column.

14. SQL to pandas

SQL → pandas

1

SELECT city, sales

```
df[["cit
y",
"sales"]
]
```

2

WHERE sales > 1000

```
df[df["s
ales"] >
1000]
```

3

ORDER BY sales DESC

```
df.sort_
values("
sales",
ascendin
g=False)
```

4

LIMIT 5

```
df.head(
5)
```

5

SELECT DISTINCT city

```
df["city
"].drop_
duplicat
es()
```

6

COUNT(*)

```
len(df)
```

7

GROUP BY region, SUM(sales)

```
df.group
by("regi
on")
["sales"
].sum()
```

8

HAVING SUM(sales) > 1000

```
df.group
by("regi
on")
["sales"
].sum().
loc[lamb
da s: s
> 1000]
```

14. SQL to pandas

SQL → pandas

9 **LEFT JOIN customers USING id**

```
df.merge
(df2,
on="cust
omer_id"
,
how="lef
t")
```

10 **UNION ALL**

```
pd.conca
t([df,
df])
```

11 **WHERE city IN (...)**

```
df[df["c
ity"].is
in(["Pun
e",
"Indore"
])]
```

12 **WHERE name LIKE '%a%'**

```
df[df["n
ame"].st
r.contai
ns("a")]
```

13 **WHERE city IS NULL**

```
df[df["c
ity"].is
na()]
```

14 **CASE WHEN ... THEN**

```
np.where
(df["sal
es"] >
1000,
"High",
"Low")
```

15 **AS (rename)**

```
df.renam
e(column
s=
{"sales"
:
"amount"
})
```

16 **ROW_NUMBER() OVER**

```
df.group
by("regi
on").cum
count()
+ 1
```

15. Excel to pandas

Excel → pandas

1

VLOOKUP

```
df.merge
(df2,
on="cust
omer_id"
,
how="lef
t")
```

2

SUMIF

```
df.group
by("regi
on")
["sales"
].sum()
```

3

COUNTIF

```
df["regi
on"].val
ue_count
s()
```

4

AVERAGEIF

```
df.group
by("regi
on")
["sales"
].mean()
```

5

IF

```
np.where
(df["sal
es"] >
1000,
"Yes",
"No")
```

6

Pivot table

```
df.pivot
_table(i
ndex="re
gion",
values="
sales",
aggfunc=
"sum")
```

7

Remove duplicates

```
df.drop_
duplicat
es()
```

8

Filter

```
df[df["c
ity"] ==
"Pune"]
```

15. Excel to pandas

Excel → pandas

9 **Sort A to Z**

```
df.sort_
values("
name")
```

10 **TRIM**

```
df["name
"].str.s
trip()
```

11 **PROPER**

```
df["name
"].str.t
itle()
```

12 **LEFT(A2, 3)**

```
df["prod
uct"].st
r[:3]
```

13 **Text to columns**

```
df["emai
l"].str.
split("@
",
expand=T
rue)
```

14 **IFERROR(x, 0)**

```
df["sale
s"].fill
na(0)
```

15 **Running total**

```
df["sale
s"].cums
um()
```

16 **RANK**

```
df["sale
s"].rank
(ascendi
ng=False
)
```

16. Errors and fixes

Error → Usually means / fix

1. **KeyError: 'Sales'**
   Column names are case sensitive. Check df.columns, leading/trailing spaces, or if the column was renamed earlier.

2. **SettingWithCopyWarning**
   Use df.loc[row_indexer, col_indexer] = value instead of chained indexing like df[df['A'] > 0]['B'] = value.

3. **ValueError on merge**
   Key column not found in one of the DataFrames, or the key columns have different dtypes. Check df.dtypes and align them before merging.

4. **Can only use .str accessor**
   The column is not text. Check df.dtypes and convert with astype(str) if needed.

5. **TypeError: unsupported operand**
   Numbers are stored as text. Convert with pd.to_numeric() before doing math.

6. **Length mismatch**
   New column has a different number of rows.

16. Errors and fixes

Error → Usually means / fix

7 **Unnamed: 0 column**

```
File saved with index
```

8 **Wrong totals after join**

```
Right table had duplicate keys
```

9 **NaN in results**

```
Blank Checks len()
```

10 **Dates sort wrong**

```
pd.to_datetime first
```

11 **MemoryError on big files**

```
Read only needed columns
```

12 **Empty DataFrame after filter**

```
Extra spaces in text
```

17. Interview questions, short answers

Question → Answer

1. **loc vs iloc?**
   loc uses labels, iloc uses positions.

2. **merge vs concat?**
   merge joins on keys, concat stacks.

3. **How do you find duplicates?**
   df.duplicated().sum(), df[df.duplicated()].

4. **apply vs vectorised code?**
   Vectorised code is faster, so use apply only when there is no built-in.

5. **What does groupby return?**
   A GroupBy object, lazy, you summarise with sum, mean, agg, etc.

6. **Handle missing values?**
   isna(), then drop, fill, or impute.

7. **count() vs size()?**
   count skips NaN, size counts every row.

17. Interview questions, short answers

Question → Answer

8 **Why reset_index()?**

```
Turns the group labels back into a column
```

9 **How to read a big CSV?**

```
Use chunksize; dtype, or
```

10 **pivot vs pivot_table?**

```
pivot_table aggregates
```

11 **transform vs agg?**

```
transform cannot shrink rows, keeps every row, agg shrinks them.
```

12 **Series vs DataFrame?**

```
One column vs a full table.
```

13 **Why inplace=True is avoided?**

```
Reassigning (df = ...) is clearer and safer.
```

14 **How to change a type?**

```
astype(), pd.to_numeric(), pd.to_datetime().
```

18. Practise these 12 tasks

Task → You will use

1. **Load a sales CSV and check blanks**
   `read_csv` `isna`
2. **Clean messy customer names**
   `str.strip` `str.title`
3. **Remove duplicate orders**
   `drop_duplicates`
4. **Revenue = qty x price**
   `new column`
5. **Sales by region and month**
   `groupby` `pivot_table`
6. **Add customer segment from another file**
   `merge`

18. Practise these 12 tasks

Task → You will use

7 **Top 3 products per region**

```
sort
_val
ues,
grou
pby
```

8 **Month-on-month growth**

```
resa
head
mple
(
pct
```

9 **Flag orders above 1000**

```
chan
np.w
ge
here
```

10 **Fix dates stored as text**

```
pd.t
o_da
teti
me
```

11 **Share of total per region**

```
tran
sfor
m
```

12 **Export a clean Excel report**

```
to_e
xcel
```

☆ THE 90-DAY SYSTEM · 5.0 BEST SELLER

Become a data analyst in 90 days.

Open it every morning. It tells you what to learn, build and ship that day. 1 hour a day, no coding background needed.

DAY 1-21	DAY 22-45	DAY 46-65	DAY 66-90
Excel + Stats	SQL	Power BI	Python + Jobs

27 portfolio projects with real data	**100+** interview Q&As with model answers
365 days of 1:1 mentor access	**10+** job applications sent by Day 90

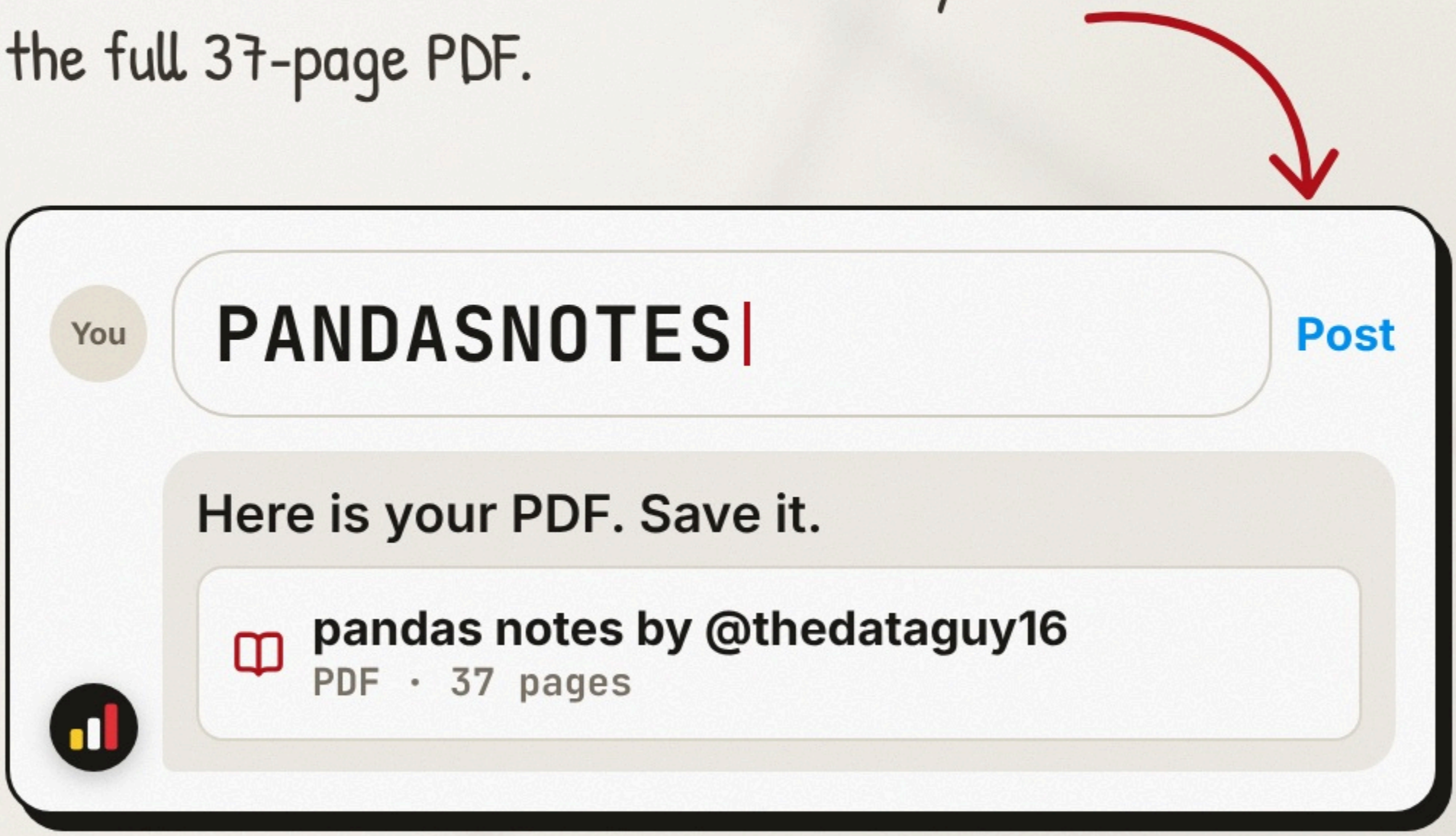
0 to job-ready, one day at a time

Built by Abhishek · 7 yrs in data · 31,000+ helped

Link in bio →

One page a day.

Comment PANDASNOTES and I'll DM you the full 37-page PDF.



Follow @thedataguy16

Send to a friend